

```

import os from dotenv import load_dotenv from openai import AzureOpenAI, APIError import
sqlite3 import pandas as pd dados_csv = pd.read_csv('dados.csv') T = pd.read_csv('tic.csv') con
= sqlite3.connect('banco.db') cursor = con.cursor() cursor.execute("""CREATE TABLE IF NOT
EXISTS vendas( venda REAL, vendedor TEXT, id TEXT ) """) con.commit()
cursor.execute('INSERT INTO vendas VALUES(?,?,?)', (1000,'Thiago',1))
cursor.execute('INSERT INTO vendas VALUES(?,?,?)', (10000,'Maria',2))
cursor.execute('INSERT INTO vendas VALUES(?,?,?)', (1220,'Fernando',3)) # crud
con.commit() cursor.execute('SELECT * FROM vendas') dados = cursor.fetchall() contexto =
f'{dados[0]}, {dados[1]}, {dados[2]}' load_dotenv(override=True) client = AzureOpenAI(
azure_endpoint=os.getenv("ENDPOINT"), api_key=os.getenv("API_KEY"),
api_version=os.getenv("API_VERSION", "2025-04-01-preview") ) deployment_name =
os.getenv("GPT5_MODEL") def validar_configuracao(): faltando = [] if not
os.getenv("ENDPOINT"): faltando.append("ENDPOINT") if not os.getenv("API_KEY"):
faltando.append("API_KEY") if not deployment_name: faltando.append("GPT5_MODEL") if
faltando: print("[Erro de configuração] As seguintes variáveis não foram definidas no .env:") for
var in faltando: print(f" - {var}") print("Preencha o arquivo .env e tente novamente.") return False
return True def main(): if not validar_configuracao(): return print("inciando ... ") while True:
pergunta = input("Digite sua pergunta ou sair: ").strip() if pergunta.lower() == "sair":
print("Atendimento finalizado. Até logo!") break if not pergunta: print("Por favor, digite uma
pergunta válida.") continue # recuperação de busca resultados_busca = T[T.apply(lambda row:
row.astype(str).str.contains(pergunta, case=False).any(), axis=1)] if not
resultados_busca.empty: dados_recuperados =
resultados_busca.head(5).to_string(index=False) else: dados_recuperados =
T.head(5).to_string(index=False) prompt = f"""\Você é um historiador especialista em titanic, use
apenas os dados que vem do dataset abaixo para responder aos usuários de forma que esteja
ensinando sempre: da {dados_recuperados} """ try: response = client.chat.completions.create(
model=deployment_name, messages=[ { "role": "system", "content": prompt, # Passando o
prompt com o RAG dinâmico }, { "role": "user", "content": pergunta } ] ) resposta_texto =
response.choices[0].message.content print(f'Resposta:{resposta_texto}') except APIError as e:
print(f'[Erro na API]: {e}') except Exception as e: print(f'[Erro inesperado]: {e}') if __name__ ==
"__main__": main()

```